

DAT410 - Design of AI Systems

Group 69, Module 1		
Teammates	Jana Leong	Chung Fung Oi
Course	Bachelor of Computer Science	
Email	janale@student.chalmers.se	funco@student.chalmers.se
No. of hours spent	15	

We hereby declare that we have both actively participated in solving every exercise. All solutions are entirely our own work, without having taken part in other solutions.

Predict the temperature

Characterisation of the problem:

When forecasting for the short-term, we believe it is best to model the problem using two distinct but complementary frameworks: as a **Dynamic System** and as a **Function Approximation**.

Under a dynamic system, we take into account temperature as the main factor. As well as other factors such as: geographic locations, humidity, precipitation, wind characteristics and atmospheric pressure. These other factors can complement the main factor in order to give us more contextual clues to better predict the temperature. It is important to note that there might be a butterfly effect, where *the other factors* greatly affect the outcome of the final predicted value, the temperature.

When modelled as a Function Approximation problem, the objective is to determine a mathematical function that maps a set of input variables to a target output, the future temperature. For this problem, the likely input variables would include historical data points (past temperature and pressure readings) and environmental features (geographical location, elevation and time of year). By analysing the statistical correlations between the input and target variables, we can then identify predictive patterns and hence estimate the future temperature.

In order to predict the temperature for the next weekend, we will also need to assume that we already know the temperature at the current space and time, location & the current weather season for both frameworks.

Temperature predictability varies significantly by geographic location. Tropical regions near the equator typically exhibit low seasonal variation, with relatively stable year-round temperatures, making forecasts more straightforward using simpler models or historical averages. In contrast, temperate and polar regions experience pronounced seasonal changes and greater short-term volatility due to shifting weather systems (e.g., cold fronts, storm tracks).

This means:

- Model selection matters: Stable climates may work well with basic statistical methods (e.g., moving averages), while variable climates benefit from dynamic system modeling that captures atmospheric evolution.
- Uncertainty differs: Predictions for equatorial regions will have narrower confidence intervals, while temperate zones require accounting for higher variability.

These geographic considerations become especially important when extending forecasts to longer horizons, such as predicting temperature for the same date next year, where seasonality and climate patterns play a dominant role.

For the next weekend

Proposed Solution

We can employ **Time Series Forecasting** to solve the problem, however specific implementation would depend on which characterisation of the problem was chosen.

Feature	Dynamic System	Function Approximation
Methodology	Recurrent Neural Network (RNN)	Moving Average Model
Characteristics of the feature	The state evolves sequentially. Memory is intrinsic and past states (data) directly affects future states (data).	No explicit state; just input features → to output predictions They exist as static functions and predictions are independent.
How it works	Maintains a running memory that evolves over time. It processes weather events in order, remembering context from previous days (e.g. A storm occurred two days ago) to influence the final prediction.	Uses a fixed window of past data to smooth out random fluctuations. It takes recent data points to map directly to a future value (e.g. Using the average temperature of the last three days to predict the forecast)
Best For	Capturing rapid changes in temperate or volatile climates where the history of the weather matters.	Stable climates or when smoothing noise is prioritized to find the underlying trend.
Subcategories (If Any)		Within the time series forecasting in this function approximation approach, we can further subdivide the solving process to forecast without a trend, forecast with a trend and forecast with trend and seasonality .

To construct a robust solution, we would use a hybrid strategy to leverage the strength of both tools. Using RNN will allow the model to recognise that a temperature rise or drop is not just an isolated number, but possibly part of a trend. This would be complimented by using Moving Average, which acts as a stabilizer by smoothing out any short-term noise to reveal the underlying seasonal trend. By combining these, we can establish a reliable baseline forecast using the Moving Average and adjust for rapid atmospheric changes using the RNN.

A crucial point to note would be that this system prioritises data relevance. We will weigh recent data more heavily while discarding statistically irrelevant data from centuries ago to minimize noise and maximize prediction accuracy.

For the same date as today but next year

If we extend the forecast to the same date next year, the approach must change, since we are no longer looking at short-term but rather long-term. Forecasting specific weather dynamics one year in advance is highly complex due to the “Butterfly Effect”. A small change in any factor today could lead to massive impacts a year from now, making the Dynamic System approach unreliable for this problem. We must rely on Function Approximation using historical statistical averages to predict the weather, as such specific atmospheric evolution is unable to be tracked over such a long duration.

While traditional methods (Moving Average) work well, machine learning can uncover less obvious, non-linear patterns that simple averaging misses:

- 1) Feature Engineering: including and incorporating additional variables such as ocean temperature (e.g. El Niño), solar cycles & atmospheric circulation patterns
- 2) Usage of regression models: Random Forests, XGBoost or Gradient Boosting can learn more complex relationships between various features.
- 3) Climate Change Adaptations: ML models can better capture non-stationary trends (e.g. accelerated warming) that violate the stationary assumption of traditional statistical averaging.
- 4) Temporal Patterns: Patterns that only occur in multi-year cycles can be identified through ML, as compared to traditional methods where these subtle differences in yearly cycles are overlooked.

Bingo lottery problem

In a bingo lottery 1000 lottery tickets are printed. Every ticket consists of a 5x5-square where each of the numbers 1 to 25 are randomly placed. A winning combination is received for a horizontal, vertical or diagonal row. For the draw of the numbers it is desired to select a set of numbers so that exactly 120 tickets give a win (or at least as close as possible), and no ticket must give more than one win. Knowledge of all the lottery tickets can be assumed.

Answer:

The following problem states:

- 1000 tickets
 - 120 winning
 - Each grid can only win with one approach — horizontally, vertically, diagonally
 - That means that there are only 12 winning lines — 5H, 5V, 2D
 - 880 losing
- 5 x 5 grid
- All ticket configurations are known

This means that the search space for the problem can go up to $2^{25} \approx 33$ million possible number combinations. Therefore to efficiently solve the problem, we should limit the search space by finding constraints.

This problem is therefore a Constraint Satisfaction Problem, that can be done with constraint programming. The goal here is to select a subset of numbers from $\{1, 2, 3, \dots, 25\}$ to draw such that exactly 120 out of 1000 tickets win.

We can model the solution in the following steps:

- 1) Defining Decision variables using, $\text{drawn}[i] \in \{0, 1\}$ for $i = 1, 2, \dots, 25$ where
 - $\text{drawn}[i] = 1$ if number i is drawn
 - $\text{drawn}[i] = 0$ if number i is not drawn
- 2) Defining Auxiliary variables
 - a) For each ticket t and each line ℓ on that ticket, $\text{line_complete}[t][\ell] \in \{0, 1\}$ where
 - i) 1 if all 5 numbers in line ℓ of ticket t are drawn
 - ii) 0 otherwise
 - b) For ticket wins has 1 complete line to be $\text{ticket_wins}[t] \in \{0, 1\}$ where
 - i) 1 if ticket t has at least one complete line
 - ii) 0 otherwise
- 3) Defining Constraints
 - a) Constraint 1: Line Completion Logic
 - i) For each ticket t and line ℓ with numbers $[n_1, n_2, n_3, n_4, n_5]$:
 $\text{line_complete}[t][\ell] = 1 \Leftrightarrow \text{drawn}[n_1] = 1 \text{ AND } \text{drawn}[n_2] = 1 \text{ AND } \dots \text{ AND } \text{drawn}[n_5] = 1$

$\text{drawn}[n_5] = 1$ meaning the line is complete if and only if all 5 numbers in it are drawn.

- b) Constraint 2: Ticket Wins if Any Line is Complete
 - i) For each ticket t : $\text{ticket_wins}[t] = 1 \Leftrightarrow \text{line_complete}[t][0] = 1 \text{ OR } \text{line_complete}[t][1] = 1 \text{ OR } \dots \text{ OR } \text{line_complete}[t][11] = 1$ meaning a ticket wins if at least 1 of its 12 lines are complete.
- c) Constraint 3: No Multiple Wins
 - i) For each ticket t : $\sum(\text{line_complete}[t][\ell] \text{ for } \ell = 0 \text{ to } 11) \leq 1$ meaning at most 1 line can be complete on any ticket, making the problem non-trivial.
- d) Constraint 4: Target 120 winners
 - i) For the sum of $\text{ticket_wins}[t]$ for 0 to 999 is 120. Mathematically, $\sum(\text{ticket_wins}[t] \text{ for } t = 0 \text{ to } 999) \approx 120$. Since exactly 120 might be difficult given the constraints no ticket wins twice then, we treat this as an optimization objective. That is to say we optimize the solution for 120, if it is impossible we take +/- of 120 for example.
- 4) Define the objective function: Minimize for 120 winners
 - a) minimize: $|\sum(\text{ticket_wins}[t]) - 120|$. We formulate this which finds the closest achievable number to 120 while satisfying all other constraints. If exactly 120 is feasible then the objective value will be 0.
 - b) Alternatively, we can minimize: $1000 \times |\sum(\text{ticket_wins}[t]) - 120| + \sum(\text{drawn}[i])$. This can help us to find the solution by breaking the tie of using the fewest numbers of draws.
 - i) This method enables us to decide on the best option (least numbers) if there exists multiple solutions to the problem.
- 5) CP solver
 - a) Constraint Propagation:
 - i) When the solver assigns $\text{drawn}[7] = 1$:
 - (1) All lines containing number 7 move closer to completion
 - (2) If a line containing 7 already has 4 other numbers drawn, that line is now complete
 - (3) This triggers: $\text{line_complete}[t][\ell] = 1$
 - (4) Which might trigger: $\text{ticket_wins}[t] = 1$
 - (5) Which might violate: "at most 1 line complete on ticket t " if another line on ticket t is already complete
 - (6) Solver immediately backtracks if violation detected
 - ii) This propagation dramatically reduces the search space.
 - b) Intelligent Search:
 - i) The CP solver doesn't randomly try combinations. It uses heuristics like:
 - (1) Variable ordering: Choose which $\text{drawn}[i]$ to assign next based on which affects the most constraints
 - (2) Value ordering: Try $\text{drawn}[i] = 1$ or 0 based on which is more likely to lead to a solution
 - (3) Conflict learning: Remember patterns that led to dead ends and avoid them

(4) Branch and bound: Prune entire branches that can't improve the objective

c) Backtracking:

i) When a partial assignment leads to:

(1) More than 120 winners already (with more numbers to draw), or

(2) Impossible to reach 120 winners (even drawing all remaining numbers), or

(3) A ticket with 2+ complete lines

ii) The solver backtracks to the most recent decision point and tries a different value.

6) Solution Algorithm

a)

1. Initialize: All drawn[i] unassigned
2. Propagate: Derive immediate consequences from constraints
3. Search:
 - a. Select an unassigned drawn[i]
 - b. Try drawn[i] = 1:
 - Propagate consequences
 - If feasible, continue recursively
 - If infeasible or can't improve objective, backtrack
 - c. Try drawn[i] = 0:
 - Propagate consequences
 - Continue as above
4. When complete assignment found:
 - Evaluate objective (deviation from 120)
 - Update best solution
 - Continue searching for better solutions
5. Return best solution found (or optimal if proven)

Public transport departure forecast

At the stops for public transport there is often a screen predicting the next departure time for each line. Suggest different approaches for how this time can be estimated.

Answer:

The problem of predicting public transport departure times can be characterised as a dynamic prediction problem within a constrained physical system. The objective is to estimate a single value, the estimated time until the next departure, based on multiple factors such as vehicle position, traffic conditions, passenger demand and infrastructure constraints. Although the output is a single value, the underlying system is complex and continuously evolving, allowing us to use multiple modelling approaches.

At a basic level, the problem can be modelled as a function approximation. Mapping would be done using a set of input variables, producing an output that represents the remaining time until departure. Typical inputs can include current GPS position, time of day, speed of vehicle, weather and historical travel times between stops. A simple approach would be to use historical averages to estimate travel time. However, this assumes stable and consistent conditions, not accounting for real-time disturbances such as congestion or passenger surges. More advanced regression techniques, such as non-parametric methods like k-NN algorithm, can help improve accuracy by identifying historically similar situations. Using these situations, we can use the outcomes to generate predictions.

Public transport systems are inherently dynamic, as the state of one vehicle influences the behaviour of others through feedback effects. For instance, if a vehicle is delayed, more passengers accumulate at the stop than planned. This leads to longer loading times and further delays. Conversely, when vehicles arrive close together, passengers may distribute across both, shortening loading time but increasing the gap to the following service. Modelling the system as a dynamic process allows these interactions to be captured over time, producing more realistic departure forecasts.

Departure predictions must also respect operational and physical constraints imposed by the transport infrastructure. Vehicles are subject to speed limits, traffic regulations, and road or track layouts may prevent overtaking. In many rail and tram systems, stops have limited capacity, often only allowing one vehicle at a time. This enforces a First-In-First-Out (FIFO) flow. As a result, the departure time of a vehicle is constrained by the speed and loading time of the vehicle ahead. This makes constraint-based modelling essential to ensure physically feasible predictions.

To further improve forecast accuracy, state estimation techniques like Kalman filtering can be used. These methods combine prior predictions from a dynamic model with new observations to produce a more reliable estimate of a vehicle's true state, allowing the forecast to adapt smoothly to sudden fluctuations or measurement errors. More advanced machine learning

approaches can further enhance departure time estimation by capturing complex, non-linear relationships that are difficult to model explicitly. Artificial Neural Networks can be used to learn hidden dependencies between factors such as weather, traffic patterns, and route-specific characteristics. Alternatively, Support Vector Machines are well suited to high-dimensional input spaces and can be used to predict delay states or classify whether a service is likely to be on time. These data-driven methods generally require large historical datasets but can outperform simpler models when such data is available.

Finally, simulation-based models can be used to represent vehicle interactions, passenger flows, and schedule adherence under varying conditions. When combined with operational control strategies such as holding policies or headway management, simulation allows both the prediction of departure times and the evaluation of interventions designed to reduce delays. Overall, while the departure forecast ultimately presents passengers with a single estimated time, it is the result of aggregating information from multiple modelling approaches.

Film festival problem

Quite a few years ago we had a system at the CSE department creating automatic schedules for film festival enthusiasts, enabling everyone to quickly order tickets before they run out. When the festival programme arrives, all films are quickly entered. Individuals can then indicate desired films with a priority and the system automatically creates an entire schedule for the week. The challenge is that it is usually impossible to see all films since many are shown at the same time, and each film is often shown several times during the week.

Answer:

The film festival problem is created due to the nature of conflicting schedules, this means that the decision-making process—the order/ timeslots of films are often in competing nature, coupled with the fact that we would ideally want to optimize for film ticket sales. Then we are limited to the films screening times, the number of cinemas available & the buyer's preferences. Therefore this is an **optimization problem with constraints**.

We, the organizer, should then choose the largest number of screenings that do not conflict with the majority of the buyer's preferences, such that everyone can purchase tickets for shows that they all would like.

Not to mention if you have rooms of different sizes then that would add more difficulty in coming up with a solution that could accommodate all the films and make all viewers happy. This problem is similar to a university having to schedule classes and rooms for various faculty and students, where each class size, room size, teaching faculty, lab availability and many other members are involved and all become constraints of the problem.

Tldr; Maximize the no. of desired films each viewer can see subjected to time conflicts, venue constraints and viewer priorities.

Alternatively, we can also view this as a **constraint satisfaction problem** since we want to optimize for user satisfaction, of which the variables are screening of each film to attend (or skip). With domains {screening_1, screening_2, ... screening_n, skip} for each film and constraints, there are no 2 films at which the user likes at overlapping times (very important for high priority votes by views).

OR in computing, we could view this as a **graph problem** as well;

Nodes = screenings of the films.

Edges = conflicting scheduling times (e.g. overlaps)

Goal = find maximum independent set

Other additional characteristics of this problem that I can think of are:

- NP hard problem: similar to weighted interval scheduling with multiple resources
- Individual vs. collective optimization: who are we optimizing for & how much of the collective?
- Hard constraints vs soft constraints: film length time conflicts vs softer constraints like viewer's priorities

Let us assume we are optimizing the solution for the collective satisfaction then we can view this problem as an **integer linear optimization problem**. We can mark each set of decisions of whether a user will watch a film as a binary decision e.g. variable x_{ij} , $x_{ij} \in \{0,1\}$; 1 if user i attends screening j , 0 otherwise. This makes it easy for the system to decide which films to keep more screenings of and keep them not overlapped. Then we can write the objective function as maximizing the sum of the priority score from the set of decisions e.g. $\sum(\text{priority}_{ij} \times x_{ij})$. Where films that are higher priority scoring are preferred when overlap occurs. The constraints would then be time conflicts, $x_{ij} + x_{ik} \leq 1$, where screenings j and k overlap. Ideally this solution is good because it can find the optimal solution, handle complex constraints easily (e.g. deconflicting, adding extra shows when needed) and can optimize for multiple users (many users in our case) simultaneously. However, this also means that this is a computationally expensive process especially when problems scale large quickly.

A quicker solution would be to use a greedy algorithm consisting of constructing a schedule via sorting by priority then assigning them to the best timeslot that does not clash with the previous schedule. Placing lesser prioritized films into the remaining timeslots such that most viewers are able to watch the things that are on their highest priority. However, the downside to this solution is that we may not find the optimal solution.

Product rating in consumer test

E.g. how determine the score for different dishwashers in a consumer test?

In the context of creating a consumer test, determining an overall score for a product can be characterised as **Prescriptive modelling**. Prescriptive modelling uses data to recommend an optimal course of action by evaluating any relevant factors and applying decision rules, resulting in a clear, actionable outcome. In this case, that outcome is a single score that will guide consumer choice.

To construct this score, weights can be assigned to various attributes of the product based on their relative importance. For example, when evaluating a dishwasher, attributes such as energy usage, water consumption, noise level, capacity, ease of use and cleaning efficiency can each be given a weighted value. A consumer who prioritises sustainability may assign higher weights to energy and water efficiency, while a household with open-plan living may place greater emphasis on noise level. Ultimately, these weights can be determined through consumer surveys, expert judgment or stated preference methods, ensuring that the model reflects real consumer priorities rather than assumptions.

Before aggregation, attribute values must be normalised to ensure comparability, since different attributes are measured on different scales (e.g. kilowatt-hours, litres or decibels). This would transform each attribute onto a common scale, such as 0 to 10, preventing any single attribute from disproportionately influencing the final score due to its unit of measurement rather than importance.

Attribute	Weight (%)	Dishwasher A	Dishwasher B
Cleaning efficacy	30	9 / 10	9 / 10
Energy usage	20	6 / 10	9 / 10
Water consumption	20	5 / 10	9 / 10
Noise level	15	8 / 10	7 / 10
Ease of use	10	7 / 10	8 / 10
Capacity	5	9 / 10	6 / 10
Total Weighted Score	100	7.25 / 10	8.45 / 10

*Scores are out of 10, where higher values indicate better performance.

Although both dishwashers demonstrate equal cleaning efficacy, Dishwasher B received a higher overall score due to its superior performance in energy and water consumption, which carry significant weight in this model. As a result, Dishwasher B may be recommended for

environmentally conscious consumers or smaller households, although it has a lower capacity than Dishwasher A.

From an optimization perspective, this problem can be framed as a weighted utility maximisation problem, where the goal is to maximise the total weighted scores across all attributes. The final objective would be to select the product with the highest score given a fixed set of weights. While the example above represents an unconstrained optimization problem, additional constraints such as budget limits, minimum performance thresholds, or regulatory requirements could be incorporated to better reflect real-world decision contexts.

In more advanced implementations, machine learning techniques can be used to refine the prescriptive model. Historical consumer ratings and purchasing behaviour can be analysed to learn optimal attribute weights or predict consumer satisfaction. These predictions can be combined with the weighted scoring framework to produce personalized recommendations, further enhancing decision quality.

Constraint satisfaction and constraint programming (read and explain)

a) Read about [constraint satisfaction problems](#)

[Links to an external site.](#) and [constraint programming](#)

[Links to an external site.](#) Use any links or material that you find useful and give them as references. Briefly explain the main concepts. For this reading task feel free to discuss with others to help you understand the topics!

(Of course this is not the only important technique, but we assume you already know some things about algorithms, optimization and data science!)

Constraint Satisfaction Problems

CSP, formally known as constraint satisfaction problem, is a set of mathematical questions of objects whose state must satisfy a number of limitations. We can think of it like solving a logical puzzle where we have rules that must be followed.

The formal definition of CSP consists of 3 components:

1. Variables (X): things we need to assign values to e.g. $X_1, X_2, \dots, X_n$
2. Domains (D): the possible values that each variable can take e.g. $D_1, D_2, \dots, D_n$
3. Constraints (C): Rules that define which combination of values are acceptable/unacceptable. There are also different type of constraints:¹
 - a. Unary Constraints apply to a single variable like “this cell cannot be 5”
 - b. Binary Constraints involve 2 variables like “these 2 cells cannot have the same number”.
 - c. Higher-order Constraints involve 3 or more variables like “each row in Sudoku must have all numbers from 1 to 9 without repetition”

Thus a solution would be one that has an assignment of values that satisfies all constraints.

There are a few types of CSPs, characterised based on their constraints and problem:

1. Binary CSPs: Problems' constraints only involve 2 variables. E.g. in a scheduling problem task A is dependent on the completion of task B, then A must be done before B.
2. Non-Binary CSPs: Problems' constraints involve >2 variables E.g. in a sitting problem, a constraint stating that A, B, and C cannot sit next 2 each other.

¹ GeeksforGeeks. (2023, June 8). *Constraint Satisfaction Problems (CSP) in Artificial Intelligence*. GeeksforGeeks. <https://www.geeksforgeeks.org/artificial-intelligence/constraint-satisfaction-problems-csp-in-artificial-intelligence/>

3. **Hard and Soft Constraints:** Hard constraints are ones that are strictly satisfied while soft constraints can be violated but at a cost. This is often used in real-world applications where not all constraints are equally important.

CSPs appear in many real-world problems such as games like sudoku where each cell is a variable, domains are numbers 1-9, and the constraint is that there are “no repeats in rows, columns and boxes”. Another example could be crossword puzzles where each cell is a variable, domains are a to z, and the constraints is that “the words have to fit the grid down and across correctly”.

Furthermore complex applications could be automated schedule planning (like the film problem above) and similar to the university planning scheduling problem. An extension of the university planning problem could be resource allocation e.g. finding what rooms to assign to what modules/ classes where there are limited resources but competing demands.

Constraint Programming

CP is the field of research where the programming paradigm is focused on solving CSPs. Unlike traditional programming where you specify how to solve a problem step-by-step, CP lets you specify what the problem is (variables, domains and constraints) and the system figures out how to solve it.

Some solution techniques are as follows:

1. **Backtracking**

Similar to traditional backtracking algorithms, it is a depth first search and recursive search algorithm that starts with no variables assigned. Pick a variable and try each possible value at a given time. If the value is consistent with the constraints, we recursively continue the search. Otherwise, we prune that branch by backtracking and try the next possible value.

We can improve backtracking by including backjumping. Backjump is simply skipping multiple variables when backtracking to quickly prune branches we know that already have broken the constraint.

Then we can also supply it with memoization of discovered inconsistencies/ consistencies. When inconsistencies are detected we can avoid repeating the search, e.g. if we know that every 3rd would have broken a constraint then we could skip the 3, 6th and 9th option.

These 2 can help improve backtracking and provide “look-ahead” scenarios where we can predict future problems before the program makes assignments.

2. **Constraint Programming**

This technique simplifies the problem by enforcing some “local consistency”:

Arc consistency (AC-3) is one of the most popular methods that is used to ensure that for every value in a variable’s domain, there exists a compatible value in related variables’ domains. This can help reduce domain sizes and detect unsatisfiability early, in turn making the problem easier to solve.

3. Local Search

LS is an iterative improvement approach that starts with a complete (by possibly inconsistent) assignment. It repeatedly modifies small parts to satisfy more constraints. Such algorithms are also min-conflict algorithms that strive to change variables that minimize constraint violations. They normally operate quickly but sometimes can appear to be incomplete as they may not find a solution even if one exists.

4. Dynamic Programming

DP is both a mathematical optimization method and a computer programming method that strives to simplify a complicated problem by breaking it down into simpler sub-problems recursively.

While some decision problems cannot be directly broken down to such sub-problems, they do still show a pattern of breaking apart recursively providing us with an optimal solution for CSPs where we can find the optimal solution to the sub-problem of its substructure through recursion. Then an optimal solution found to the sub-problems of a bigger problem is said to have optimal substructure.

5. Hybrid Approaches

Lots of modern solutions use a hybrid strategy consisting of multiple techniques such as the Very Large Neighborhood Search (VLNS) uses structured search and propagations capabilities of CP with the efficiency of local search methods to solve complex, large scale combinatorial optimization problems.

Since the neighbourhood is very hard it makes it impossible to search by brute force, instead the search is done by solving a sub-problem, often using CP to find the best neighbour in that large region. This can be very useful in areas such as network resource allocation where the computer network is so large and so many packets have to go around that VLNS can depend on partial tree searches to navigate the neighbourhood, avoiding a local optima but small enough selection of a neighborhood that can be solved quickly, using an iterative process to choose a neighbourhood (& possibly optimize it with CP) and moving to a new solution repeating till termination conditions are met.

Resource-Constrained project scheduling problems and extensions of them are also other possible applications of VLNS with CP.

Benefits of CSPs in AI

CSPs in AI can provide a clear and structured way to define problems using variables, possible values and rules making them easily predictable which helps AI clearly identify patterns earlier, since input format is similar. These definitions help smart search techniques like backtracking (and forward-checking, another CP approach) reduce time needed to find solutions, offering flexibility as they can be used without the need for expert knowledge.

Challenges in Solving CSPs

A major downside to CSPs are if there are too many variables, domains and constraints, the problem greatly increases in complexity and finding a solution can take too long & require much computing power. In the worst case scenario, we might not find a solution. In such a case we might need to adjust constraints and accept compromises in order to find an acceptable solution. Furthermore, in real life applications, problems are often dynamic in nature, constantly changing constraints, which means that CSP solutions need to be updated often.

b) (voluntary) Is it possible to generalize *constraint programming (and constraint propagation)* to numbers, and not just true and false? Think and describe your thoughts. *(Explanation: In constraint programming we use a table - or more generally a function - to describe a constraint. For example, if we have two three-valued variables, the constraint can be described by a table with $3 \times 3 = 9$ values, all of which is either T or F. But what would happen if we replaced T and F by numbers of some kind? Another completely different issue is if you can have real numbers as variables in constraint programming and how to handle that - this we not what I intended to ask about but if this is how you interpreted the question, that's fine - this is also an interesting issue.)*